

Транзакции

Под транзакцией понимается некоторая группа DML команд. Все изменения сделанные ими, сохраняются в отдельной области памяти до окончательного подтверждения изменений (успешное завершение транзакции), либо до их отмены. Если во время транзакции делается запрос на выборку данных, то создается отдельное представление.

Для чего нужны транзакции?

В англоязычной литературе концепция транзакций описывается аббревиатурой ACID. Эта аббревиатура расшифровывается как *Atomicity*, *Consistency*, *Isolation* и *Durability* (атомарность, непротиворечивость, изолированность и долговечность). Это тесно связанные критерии, которым должна соответствовать правильно функционирующая система транзакционной обработки:

Атомарность	Транзакция должна функционировать как единая неделимая единица работы таким образом, чтобы вся транзакция была либо выполнена, либо отменена. Когда транзакции являются атомарными, не существует такого понятия, как частично выполненная транзакция: все или ничего.
Непротиворечивость	База данных должна всегда переходить из одного непротиворечивого состояния в последующее. Например, в банковской системе может существовать требование равенства суммы, списываемой с одного счёта, сумме, зачисляемой на другой. Это бизнес-правило и оно не может быть гарантировано только проверками целостности, его должны соблюдать программисты при написании кода транзакций. Если какая-либо транзакция произведёт списание, но не произведёт зачисление, то система останется в некорректном состоянии и свойство согласованности будет нарушено. Ещё одно замечание касается того, что в ходе выполнения транзакции согласованность не требуется. В примере, списание и зачисление будут, скорее всего, двумя разными подоперациями и между их выполнением внутри транзакции будет видно несогласованное состояние системы. Однако не нужно забывать, что при выполнении требования изоляции, никаким другим транзакциям эта несогласованность не будет видна. А атомарность гарантирует, что транзакция либо будет полностью завершена, либо ни одна из операций транзакции не будет выполнена. Тем самым

	эта промежуточная несогласованность является скрытой.
<i>Изолированность</i>	Результаты транзакции обычно невидимы другим транзакциям, пока она не закончена, т.е. во время выполнения транзакции параллельные транзакции не должны оказывать влияние на её результат. Это свойство не соблюдается на уровне изолированности Read Committed и ниже.
<i>Долговечность</i>	Будучи зафиксированы, внесенные в ходе транзакции изменения становятся постоянными. Это означает, что изменения должны быть записаны так, чтобы данные не могли быть потеряны в случае сбоя системы. Долговечность, однако, является несколько расплывчатой концепцией, поскольку на самом деле существует много уровней. Некоторые стратегии обеспечения долговечности предоставляют более сильные гарантии безопасности, чем другие, и ни одна из них не является надежной на 100%.

Синтаксис DCL команд

Полный синтаксис DCL команд:

```
begin {transaction | tran} [transaction_name]
save {transaction | tran} savepoint_name
commit {transaction | tran | work} [transaction_name]
rollback {transaction | tran | work} [transaction_name | savepoint_name]
```

transaction_name	Это имя, назначенное транзакции. Оно должно соответствовать правилам создания идентификаторов.
transaction tran work	Это синонимы. Но, <code>transaction</code> и <code>tran</code> - T-SQL расширения, <code>work</code> – ключевое слово, утвержденное SQL стандартом.
commit	Эта команда фиксирует все сделанные изменения.
rollback	Эта команда отменяет текущую транзакцию в любое время до вызова <code>commit tran</code> .
savepoint	Это специальный маркер внутри транзакции. Используя этот маркер, созданный с помощью команды <code>save transaction</code> , можно в рамках транзакции отменить некоторые изменения из заданного набора команд и закрепить остальные.

Если при вызове **ROLLBACK TRAN** не указано ни имя транзакции, ни имя **savepoint**, будут отменены действия **всех** операторов до первого **BEGIN TRAN** в пакете.

COMMIT TRANSACTION, ROLLBACK TRAN и SAVE TRAN, вызванные вне открытой транзакции, не выполняет никаких действий и не выдают сообщения об ошибке.

Любые модификации данных в рамках транзакции носят временный характер до тех пор, пока транзакция не закрыта. Состояние данных перед COMMIT или ROLLBACK предусматривает следующее:

1. В лог транзакций записывается маркер начала транзакции
2. Результаты DML операций записываются в буфер данных сервера, в лог транзакций заносится предыдущее состояние изменяемых данных.
3. Пользователь может просмотреть совершенные изменения с помощью обычных запросов
4. Доступ к измененным данным других пользователей определяется выбранным уровнем

изоляции

5. Модифицированные записи блокируются.

Если транзакция прошла успешно то команда COMMIT фиксирует все сделанные изменения. Состояние данных после вызова COMMIT предусматривает следующее:

1. Измененные данные записываются на диск.
2. Предыдущее состояние теряется, из лога транзакций удаляется информация о внесенных изменениях (зависит от настроек базы данных).
3. Данные становятся общедоступными
4. Освобождаются все блокировки.

Пример :

```
BEGIN TRAN
UPDATE suser SET status = 0 WHERE user_login = 'PT27'
SELECT * FROM suser WHERE user_login = 'PT27'
COMMIT TRAN
```

Отменить изменения данных, внесенные с начала транзакции, можно с помощью команды ROLLBACK. После вызова:

1. С помощью лога транзакций отменяются все изменения данных, совершенные с момента начала данной транзакции
2. Восстанавливаются предыдущие значения
3. Снимаются все блокировки

Пример:

```
BEGIN TRAN
DELETE FROM suser
ROLLBACK TRAN
```

Используя команду SAVE TRANSACTION можно в рамках текущей транзакции создать именованный маркер, который разделит ее на секции. В этом случае возможно отменить не все сделанные в рамках данной транзакции изменения, а только их часть, начиная с созданного маркера. Механизм данной отмены аналогичен механизму отмены транзакции.

Пример :

```
BEGIN TRAN
INSERT INTO suser(user_login,status) VALUES ('PT27',-1)
SAVE TRAN INS_SAVEPOINT
DELETE FROM suser WHERE user_login = 'PT27'
ROLLBACK TRAN INS_SAVEPOINT
```

```
UPDATE suser SET status = 0 WHERE user_login = 'PT27'  
COMMIT TRAN
```

Две опции характеризуют работу с транзакциями:

1. Режим транзакции, определяющий их явный или неявный способ открытия
2. Уровень изоляции, определяющий степень закрытости данных на время транзакции

Режим транзакции

Sybase ASE поддерживает следующие режимы транзакций:

Unchained	По умолчанию устанавливается режим, называемый несвязанным (unchained) или режимом Transact-SQL, в котором для начала транзакции требуется явное указание оператора begin transaction , а для ее окончания требуется указать соответствующий (парный) оператор commit transaction или rollback transaction
Chained	Можно установить режим, который в стандарте SQL называется связанным (chained), когда транзакция начинается неявно при выполнении любого оператора выбора или модификации данных. К этим операторам относятся: delete, insert, open, fetch, select и update . Но для окончания транзакции здесь нужно явно указать оператор commit transaction или rollback transaction .

Можно устанавливать любой режим транзакции, используя опцию **chained** команды **set**. Не рекомендуется смешивать эти два режима, поскольку поведение хранимых процедур и триггеров может меняться в зависимости от выбранного режима.

***Замечание.** SET chained нельзя вызывать внутри открытой транзакции.*

Пример:

```
SET chained on      -- chained mode
SET chained off     -- unchained mode
```

В **chained** режиме, ASE неявно открывает транзакцию перед выполнением команд SELECT, DELETE, INSERT или UPDATE. В зависимости от выбранного режима при выполнении одного и того же набора операторов могут быть получены различные результаты.

Для определения текущего режима транзакции ASE можно воспользоваться глобальной переменной **@@tranchained**. **@@tranchained** возвращает 0 для **unchained** и 1 для **chained** режима транзакций.

Пример:

```
INSERT INTO SUSER VALUES ('BM06', 'Петренко', 'Петр', 'Петрович', 0)
BEGIN TRAN
DELETE FROM SUSER WHERE user_login = 'BM06'
ROLLBACK TRAN
```

В **unchained** режиме команда ROLLBACK воздействует только на команду DELETE, так что по окончании скрипта, пользователь «BM06» все еще находится в таблице SUSER.

В **chained** режиме, команда INSERT неявно начинает транзакцию, и ROLLBACK воздействует на все команды до начала этой транзакции, включая и сам INSERT.

Явные и неявные транзакции

В зависимости от способа интерпретации SQL команд и выбранного режима транзакций, можно выделить неявную, явную и ANSI – совместимую модель поведения ASE.

Неявные транзакции возникают, если при выполнении DML команд явно не указана команда BEGIN TRANSACTION. ASE в этом случае выполняет каждую операцию в рамках отдельной транзакции. Явное указание команды COMMIT TRANSACTION не требуется. Например, следующие команды интерпретируются как три независимые транзакции:

```
1> insert ...
2> go
1> insert ...
2> go
1> insert ...
2> go
```

Размещением DML операторов между командами BEGIN TRANSACTION и commit transaction определяет явную транзакцию. Число явных транзакций определяется числом независимых пар команд BEGIN TRANSACTION и COMMIT TRANSACTION . Например, следующие команды интерпретируются как одна транзакция:

```
1> begin transaction
2> insert ...
3> insert ...
4> insert ...
5> commit transaction
6> go
```

В ANSI совместимой модели транзакция открывается перед DML командой или командой выборки. Но для ее закрытия обязательно нужно использовать команду COMMIT TRANSACTION . Число транзакций определяется числом команд COMMIT TRANSACTION. Например, следующие команды интерпретируются как одна транзакция:

```
1> insert ...
2> insert ...
3> insert ...
4> commit transaction
5> go
```


Вложенные транзакции

Транзакции допускают вложенность. При этом вложенные предложения BEGIN TRANSACTION и COMMIT TRANSACTION только увеличивают и уменьшают внутренний счетчик транзакций. Фактически открытие и закрытие транзакции определяется положением самой внешней пары **begin .../ commit...** .

ASE не закатывает транзакцию до тех пор, пока не встретит команду COMMIT TRANSACTION, которая соответствует самой первой BEGIN TRANSACTION, с которой транзакция началась. Обычно, «вложение» транзакций происходит при вызове хранимых процедур или триггеров, которые самостоятельно управляют транзакциями.

Глобальная переменная @@trancount позволяет определить актуальный уровень вложенности транзакций. При явном или неявном открытии транзакции значение этой переменной устанавливается в 1. Каждый последующий вызов BEGIN TRANSACTION, увеличивает, а COMMIT TRANSACTION уменьшает @@trancount на 1. Срабатывание триггера также увеличивает @@trancount, и транзакция начинается с команды, которая вызвала срабатывание триггера.

Пример:

begin tran select @@trancount	/* @@trancount = 1 */
begin tran select @@trancount	/* @@trancount = 2 */
begin tran select @@trancount	/* @@trancount = 3 */
commit tran select @@trancount	/* @@trancount = 2*/
commit tran select @@trancount	/* @@trancount = 1 */
commit tran select @@trancount	/* @@trancount = 0 */

Только в **unchained** режиме с помощью явного вызова BEGIN TRANSACTION можно получить вложенные транзакции.

В **chained** режиме, при неявном открытии транзакции, дальнейшие команды выборки или модификации данных, не открывают ни новых, ни вложенных транзакций до тех пор, пока не закроется транзакция, частью которой они являются. В chained режиме, транзакция открывается при вызове оператора выборки или модификации данных независимо от того выполнен этот

оператор успешно или нет.

Пример:

```
begin transaction
begin transaction
    insert into PUBLISHERS values ('BM05','Петренко','Петр',
                                   'Петрович',0)
    insert into PUBLISHERS values ('BM06','Кравченко','Александр',
                                   'Иванович',0)

commit transaction
commit transaction
```

В **chained** режиме первый вызов COMMIT TRANSACTION делает все изменения постоянными и закрывает транзакцию. Второй вызов избыточен.

Уровни изолированности транзакций

Уровень изолированности транзакций — значение, определяющее уровень, при котором в транзакции допускаются несогласованные данные, то есть степень изолированности одной транзакции от другой. Более высокий уровень изолированности повышает точность данных, но при этом может снижаться количество параллельно выполняемых транзакций. С другой стороны, более низкий уровень изолированности позволяет выполнять больше параллельных транзакций, но снижает точность данных.

При параллельном выполнении транзакций возможны следующие проблемы:

- **«грязное» чтение** (уровень `READ UNCOMMITTED`) — чтение данных, добавленных или изменённых транзакцией, которая впоследствии не подтвердится (откатится). На уровне изоляции `READ UNCOMMITTED` транзакции могут видеть результаты незафиксированных транзакций, что может создать множество проблем.

Пример:

Предположим, имеются две транзакции, открытые различными приложениями, в которых выполнены следующие SQL-операторы:

Транзакция 1	Транзакция 2
<code>SELECT f2 FROM tbl1 WHERE f1=1;</code>	
<code>UPDATE tbl1 SET f2=f2+1 WHERE f1=1;</code>	
	<code>SELECT f2 FROM tbl1 WHERE f1=1;</code>
<code>ROLLBACK WORK;</code>	

В транзакции 1 изменяется значение поля `f2`, а затем в транзакции 2 выбирается значение этого поля. После этого происходит откат транзакции 1. В результате значение, полученное второй транзакцией, будет отличаться от значения, хранимого в базе данных.

- **неповторяющееся чтение** (уровень `READ COMMITTED`) — при повторном чтении в рамках одной транзакции, ранее прочитанные данные оказываются изменёнными или удалёнными. Это уровень изоляции по умолчанию для большинства СУБД. Он удовлетворяет вышеприведенному простому определению изолированности: транзакция увидит только те изменения, которые были уже зафиксированы другими транзакциями к моменту ее начала, а произведенные ею изменения останутся невидимыми для других транзакций, пока текущая транзакция не будет зафиксирована. На этом уровне возможен феномен невозпроизводимого чтения (*nonrepeatable read*). Это означает, что вы можете выполнить одну и ту же команду дважды и получить различный результат.

Пример:

Предположим, имеются две транзакции, открытые различными приложениями, в которых выполнены следующие SQL-операторы:

Транзакция 1	Транзакция 2
SELECT f2 FROM tbl1 WHERE f1=1;	SELECT f2 FROM tbl1 WHERE f1=1;
UPDATE tbl1 SET f2=f2+1 WHERE f1=1;	
COMMIT;	
	SELECT f2 FROM tbl1 WHERE f1=1;

В транзакции 2 выбирается значение поля f2, затем в транзакции 1 изменяется значение поля f2. При повторной попытке выбора значения из поля f2 в транзакции 2 будет получен другой результат. Эта ситуация особенно неприятна, когда данные считываются с целью их частичного изменения и обратной записи в базу данных.

- **фантомное чтение** (уровень REPEATABLE READ) — при повторном чтении в рамках одной транзакции прочитаны данные(новые "фантомные" строки), которых при предыдущих чтениях не было, хотя они удовлетворяли условиям отбора предыдущего чтения. Это уровень изоляции решает проблемы, которые возникают на уровне *READ COMMITTED*. Он гарантирует, что любые строки, которые считываются в контексте транзакции, будут «выглядеть такими же» при последовательных операциях чтения в пределах одной и той же транзакции, однако теоретически на этом уровне возможен феномен фантомного чтения (*phantom reads*). Попросту говоря, фантомное чтение может происходить в случае, если в одной транзакции выбирается некоторый диапазон строк, затем другая транзакция вставляет новую строку в этот диапазон, после чего в первой транзакции выбирается тот же диапазон снова. В результате первая транзакция увидит новую «фантомную» строку.

Пример:

Предположим, имеется две транзакции, открытые различными приложениями, в которых выполнены следующие SQL-операторы:

Транзакция 1	Транзакция 2
	SELECT SUM(f2) FROM tbl1;
INSERT INTO tbl1 (f1,f2) VALUES (15,20);	
COMMIT;	
	SELECT SUM(f2) FROM tbl1;

В транзакции 2 выполняется SQL-оператор, использующий все значения поля f2. Затем в транзакции 1 выполняется вставка новой строки, приводящая к тому, что повторное выполнение SQL-оператора в транзакции 2 выдаст другой результат. Такая ситуация называется фантомным чтением. От неповторяющегося чтения оно отличается тем, что результат повторного обращения к данным изменился не из-за изменения/удаления самих этих данных, а из-за появления новых (фантомных) данных.

- **упорядочиваемость** (Serializable) - Самый высокий уровень изолированности; транзакции полностью изолируются друг от друга. На этом уровне результаты параллельного выполнения транзакций для базы данных в большинстве случаев можно считать совпадающими с последовательным выполнением тех же транзакций (по очереди в каком-либо порядке). Это самый высокий уровень изоляции, и решает проблему фантомного чтения, заставляя транзакции выполняться в таком порядке, чтобы исключить возможность конфликта. В двух словах, уровень *SERIALIZABLE* блокирует каждую строку, которую транзакция читает. На этом уровне может возникать множество задержек и конфликтов при блокировках. На практике данный уровень изоляции применяется достаточно редко, но потребности вашего приложения могут заставить вас использовать его, согласившись с меньшей степенью совместного доступа в пользу стабильности данных.

Уровни и схемы блокировок

Уровень блокировки в базе данных определяет, сколько данных блокируется за один раз.

Теоретически сервер баз данных может блокировать от всей базы данных до одного столбца. Но такие крайности могут, с одной стороны, существенно уменьшить возможности одновременной работы с базой данных (число пользователей, которые могут обратиться к данным), а с другой – увеличить затраты на обработку блокировок на сервере. Adaptive Server поддерживает блокировки на уровне таблицы, страницы и строки.

Чем выше уровень блокировок, тем меньший объем работы требуется для установки блокировок и управления ими. Если запрос должен прочитать или обновить много строк в таблице, он может сделать следующее:

- установить одну блокировку на уровне таблицы.
- блокировать каждую страницу, содержащую требуемые строки.
- блокировать каждую строку.

Для обслуживания блокировки на уровне таблицы нужно меньше системных ресурсов, но блокировки на высоких уровнях могут привести к снижению производительности, поскольку другим пользователям придется ждать снятия блокировок. Чем ниже уровень блокировки, тем больше данных доступно другим пользователям. Однако блокировки на низких уровнях могут привести к снижению производительности, так как потребуются больше затрат для обслуживания и координации возросшего количества блокировок. Чтобы обеспечить наилучшую производительность, схема блокировки должна учитывать и требования, предъявляемые к одновременному доступу, и системные затраты.

Adaptive Server поддерживает следующие схемы блокировки:

- Блокировка всех страниц (**allpages locking**), при которой блокируются страницы данных и индекса.
- Блокировка страниц данных (**datapages locking**), при которой блокируются только страницы данных.
- Блокировка строк данных (**datarows locking**), при которой блокируются только строки данных.

Для любой схемы блокировки сервер Adaptive Server может заблокировать всю таблицу, если запрос заблокировал много страниц или строк, или только страницы и строки, которые должны быть считаны запросом.

Блокировка всех страниц

При блокировке всех страниц блокируются как страницы данных, так и страницы индекса. Когда происходит обновление значения в строке таблицы с блокировкой всех страниц, на соответствующую страницу данных устанавливается монополярная блокировка. Монополярная блокировка также устанавливается на все страницы индекса, изменяемые операцией обновления. Эти блокировки являются транзакционными, то есть удерживаются до завершения транзакции.

Во многих случаях проблемы при одновременном доступе к данным, вызываемые блокировкой всех страниц, происходят из-за блокирования страниц индекса, а не самих страниц данных. У страниц данных строки длиннее, чем у индексов, и часто этих строк на странице немного. Если у индекса короткие ключи, страница индекса может хранить от 100 до 200 ключей. Монополярная блокировка страницы индекса может не давать другим пользователям обратиться к строкам, на которые ссылается эта страница, причем таких строк может быть гораздо больше, чем на заблокированной странице данных.

Блокировка страниц данных

При блокировке страниц данных по-прежнему целиком блокируются страницы данных, но страницы индекса не блокируются. Если требуется изменить строку на странице данных, эта страница блокируется и блокировка удерживается до завершения транзакции. Обновления страниц индекса выполняются с помощью защепок, которые не являются транзакционными. Защелки удерживаются только на время выполнения физического изменения страницы и затем немедленно снимаются. Записи на страницах индекса блокируются неявно при блокировке страниц данных. Для страниц индекса не применяются транзакционные блокировки.

Блокировка строк данных

При блокировании строк данных блокировки устанавливаются на отдельные строки на страницах данных. Строки и страницы индекса не блокируются. Если требуется изменить строку на странице данных, для страницы устанавливается нетранзакционная защелка. Защелка действует, пока на странице выполняется физическое изменение данных, а затем снимается. Блокировка строки данных удерживается до завершения транзакции. Строки индекса обновляются с помощью защепок на странице индекса, но не блокируются. При блокировке строки данных записи индекса блокируются неявным образом.

Блокировки страниц и строк

Для страниц и строк могут устанавливаться следующие блокировки:

Разделяемые (SHARED)	Сервер Adaptive Server использует разделяемые блокировки для операций чтения. Если разделяемая блокировка применена к странице данных, строке данных или странице индекса, то другие транзакции также могут установить на эти данные разделяемую блокировку, даже если первая транзакция еще не закончила работу. Однако ни одна транзакция не может установить
-----------------------------	---

	монопольную блокировку страницы или строки до тех пор, пока с этой страницы или строки не будут сняты все разделяемые блокировки. Это значит, что несколько транзакций могут одновременно читать страницу или строку, но ни одна транзакция не может менять находящиеся там данные, пока действует разделяемая блокировка. Транзакции, которым нужна монопольная блокировка, ждут снятия разделяемых блокировок. По умолчанию сервер Adaptive Server снимает разделяемые блокировки после завершения сканирования страницы или строки. Разделяемые блокировки не удерживаются до завершения оператора или транзакции, если пользователь не потребовал этого.
Монопольные (EXCLUSIVE)	Сервер Adaptive Server применяет монопольные блокировки для операций изменения данных. Когда транзакция устанавливает монопольную блокировку, другие транзакции не могут установить никакую блокировку страницы или строки до тех пор, пока первая транзакция не завершится и не снимет монопольную блокировку. Они приостанавливают работу и ждут снятия монопольной блокировки.
Блокировки обновления (UPDATE)	Сервер Adaptive Server применяет блокировки обновления на начальном этапе выполнения операторов update, delete или fetch (для курсоров, в объявлении которых есть инструкция for update) при чтении страницы или строки. К страницам или строкам, на которые установлена блокировка обновления, могут применяться разделяемые блокировки, но не другие блокировки обновления или монопольные блокировки. Блокировки обновления помогают избежать взаимоблокировок и конфликтов блокировок. Если нужно изменить страницу или строку, блокировка обновления повышается до монопольной блокировки, как только с этой страницы или строки будут сняты все остальные разделяемые блокировки.

Общее правило таково, что операции чтения устанавливают разделяемые блокировки, а операции записи – монопольные блокировки. Adaptive Server применяет монопольные блокировки и блокировки обновления на уровне страниц и строк только в случае, если столбец, указанный в аргументе поиска, входит в индекс. Если ни по каким столбцам-аргументам поиска нет индексов, то Adaptive Server должен установить блокировку на уровне таблицы.

Блокировки таблицы

Для таблиц могут устанавливаться следующие блокировки:

Блокировки занятости (INTENT)	Блокировка занятости указывает, что для таблицы в настоящее время действуют блокировки строк или страниц. Adaptive Server устанавливает для таблицы блокировку каждый раз, когда устанавливается разделяемая или монопольная блокировка страницы или строки. Таким образом, блокировка занятости может быть монопольной или разделяемой. Блокировка занятости не дает другим транзакциям устанавливать конфликтующие блокировки уровня таблицы для таблицы, содержащей заблокированную страницу. Блокировка занятости действует, пока в транзакции действуют блокировки страниц или строк.
Разделяемые (SHARED)	Эта блокировка похожа на разделяемую блокировку страницы или строки, но она устанавливается для всей таблицы. Например, сервер Adaptive Server устанавливает разделяемую блокировку таблицы для выполнения команды select, содержащей инструкцию holdlock, если эта команда не использует индекс. Команда create nonclustered index также устанавливает разделяемую блокировку таблицы.
Монопольные (EXCLUSIVE)	Эта блокировка похожа на разделяемую блокировку страницы или строки, но она устанавливается для всей таблицы. Например, сервер Adaptive Server устанавливает монопольную блокировку таблицы на время выполнения команды create clustered index. Операторы update и delete требуют монопольных блокировок таблицы, если столбцы в их аргументах поиска не индексируются. Монопольные блокировки таблиц устанавливаются и во время выполнения операторов select into (в том числе и тех, которые создают временные таблицы с помощью конструкции tempdb..имя_таблицы). Таблицы, созданные с помощью конструкции #имя_таблицы, могут использоваться только создавшим их процессом и не блокируются.

Указание схемы блокировки для таблицы

Adaptive Server позволяет пользователю самому выбрать наилучшую схему блокировки для каждой таблицы в приложении и адаптировать схему блокировки для таблицы, если конкуренция или производительность требует каких-либо изменений. Схемы блокировки задаются с помощью следующих инструментов:

процедура sp_configure	задает схему блокировки по умолчанию для всего сервера
команда create table	задает схему блокировки для создаваемых таблиц
команда alter table	изменяет схему блокировки для таблицы на другую схему блокировки
команда select into	задает схему блокировки для таблицы, создаваемой из набора результатов запроса к другим таблицам

Указание схемы блокировки для всего сервера

Параметр конфигурации **lock scheme** задает схему блокировки, которая будет использоваться для новых таблиц, при создании которых (командой `create table`) схема блокировки не указана.

Для просмотра текущей схемы блокировки выполните следующую процедуру:

```
sp_configure "lock scheme"
```

Возможный вывод:

Parameter Name	Default	Memory Used	Config Value	Run Value
-----	-----	-----	-----	-----
lock scheme	allpages	0	datarows	datarows

Синтаксис изменения схемы блокировки:

```
sp_configure "lock scheme", 0,{allpages | datapages | datarows}
```

Следующая команда устанавливает блокировку страниц данных как схему блокировки по умолчанию для всего сервера:

```
sp_configure "lock scheme", 0, datapages
```

После того как Adaptive Server был установлен, схема блокировки изначально устанавливается в `allpages` (блокировка всех страниц).

Указание схемы блокировки с помощью команды `create table`

В команде `create table` можно задать схему блокировки для создаваемой таблицы. Синтаксис этого определения следующий:

```
create table имя_таблицы (список_столбцов)
[lock {datarows | datapages | allpages}]
```

Если для таблицы не указывается схема блокировки, используется схема блокировки по умолчанию для сервера, установленная параметром конфигурации `lock scheme`.

Следующая команда задает для таблицы `new_publishers` блокировку строк данных:

```
create table new_publishers(
  pub_id char(4) not null,
  pub_name varchar(40) null,
  city varchar(20) null,
  state char(2) null
) lock datarows
```

Схема блокировки, указанная в команде `create table`, переопределяет установку по умолчанию для всего сервера.

Изменение схемы блокировки с помощью команды `alter table`

Для изменения схемы блокировки для таблицы используется команда `alter table`. Синтаксис:

```
alter table имя_таблицы lock {allpages | datapages | datarows}
```

Следующая команда изменяет схему блокировки для таблицы `titles` на блокировку строк данных:

```
alter table titles lock datarows
```

В команде `alter table` в качестве новой схемы блокировки можно указать любую схему блокировки, отличную от текущей. Для замены блокировки всех страниц на блокировку только данных необходимо скопировать строки данных на новые страницы и заново создать все индексы в таблице.

Эта операция состоит из нескольких шагов, и для нее необходимо пространство для копирования таблицы и индексов. Время, требующееся для ее выполнения, зависит от размера таблицы и количества индексов.

Для замены блокировки страниц данных на блокировку строк данных (или наоборот) не требуется

копировать страницы данных и перестраивать индексы. Переключение между схемами блокировки только данных изменяет только системные таблицы и выполняется за несколько секунд.

Указание схемы блокировки в команде **select into**

При создании новой таблицы командой **select into** также можно задать схему блокировки.

Синтаксис этого определения следующий:

```
select список_выборки
into [[база_данных.]владелец.]имя_таблицы
lock {datarows | datapages | allpages}
from ...
```

Если в команде **select into** не указывается схема блокировки, то для новой таблицы используется схема блокировки по умолчанию на уровне сервера, определенная параметром конфигурации **lock scheme**.

Следующая команда задает для создаваемой таблицы блокировку строк данных:

```
select title_id, title, price
into bus_titles
lock datarows
from titles
where type = "business"
```

Временные таблицы с именем в форме **#имя_таблицы**, являются однопользовательскими таблицами, поэтому конкуренции в данном случае не возникает. Для временных таблиц, которые могут совместно использоваться несколькими пользователями (то есть созданных в базе данных **tempdb**), могут использоваться любые схемы блокировки.

В транзакциях можно явным образом блокировать таблицу с помощью команды **lock table**. Это нужно в следующих случаях:

- Когда нужно немедленно заблокировать всю таблицу, не дожидаясь повышения блокировки.
- Когда в запросах или транзакциях используется несколько сканирований, и ни одно из сканирований не блокирует столько страниц или строк, сколько нужно для повышения блокировки, но общее число блокировок очень велико.
- Если к большим таблицам (особенно таблицам с блокировкой строк данных) нужно обращаться, используя уровни изоляции транзакций 2 или 3, а блокировка не может быть повышена из-за других задач. Использование команды **lock table** позволяет не допустить,

чтобы доступные блокировки закончились.

Блокировки таблиц снимаются в конце транзакции.

Управление уровнями изоляции

Уровень изоляции транзакций, используемый командами `select`, устанавливается следующим образом:

- для всех запросов в сеансе – команда **set transaction isolation level**;
- для отдельного запроса – инструкция **at isolation**;
- для определенных таблиц в запросе – ключевые слова **holdlock**, **noholdlock** и **shared**.

При выборе уровней изоляции для приложений следует использовать минимальный уровень блокировки, согласующийся с вашей бизнес-моделью. Установка уровня изоляции для сеанса в сочетании с установкой режимов блокировки на уровне запросов позволяет одновременно выполняющимся транзакциям достигать необходимых результатов с минимальными блокировками.

Установка уровней изоляции для сеанса

В стандарте SQL ANSI-92 уровнем изоляции по умолчанию является уровень 3. Для Sybase ASE уровень изоляции по умолчанию - 1. Для указания уровня изоляции в Transact-SQL используется команда `set transaction isolation level`. Например, следующая команда делает уровень 3 уровнем изоляции по умолчанию для текущего сеанса:

```
set transaction isolation level 3
```

Если для сеанса указан уровень изоляции 3, можно заставить запрос выполняться на уровне 1 с помощью ключевого слова **noholdlock**.

Если уровнем изоляции по умолчанию в Adaptive Server является уровень 1 или если с помощью команды `set transaction isolation level` указан уровень 0 или 2, можно задать уровень 3 с помощью параметра **holdlock**, чтобы удерживать разделяемые блокировки до конца транзакции.

Текущий уровень изоляции для сеанса можно определить из глобальной переменной **@@isolation**.

В операторе `select` можно указывать параметры **holdlock**, **noholdlock** и **shared**, которые применяются к отдельным таблицам, и инструкцию `at isolation`, которая применяется ко всему запросу.

Использование ключевых слов **holdlock**, **noholdlock** и **shared**

Уровень блокировки сеанса можно переопределять, указав параметры **holdlock**, **noholdlock** или **shared** для отдельных таблиц в командах `select` и `readtext`:

Обеспечиваемый уровень изоляции	Ключевое слово	Действие
1	noholdlock	Не удерживает блокировки до конца транзакции. Используется, чтобы при уровне 3 выполнить команду с уровнем изоляции 1
2, 3	holdlock	Удерживает разделяемые блокировки до завершения транзакции. Используется, чтобы при уровне 1 выполнить команду с уровнем изоляции 3
0	shared	Применяет разделяемые блокировки, а не блокировки обновления для операторов select в курсорах, открытых для обновления

Эти ключевые слова влияют на блокировку транзакции:

- Если указано ключевое слово holdlock, все блокировки удерживаются до конца транзакции.
- Если в запросе указывается ключевое слово holdlock, а для сеанса установлен уровень изоляции 0, то Adaptive Server выводит предупреждение и игнорирует инструкцию holdlock, не устанавливая блокировки на время выполнения запроса.
- Если указываются параметры holdlock и read uncommitted, Adaptive Server выводит сообщение об ошибке и запрос не выполняется.

Примеры:

Установка уровня изоляции и запрос	Уровень изоляции
<pre>set transaction isolation level 2 SELECT * FROM suser holdlock at isolation read committed</pre>	/* 3 уровень изоляции */
<pre>set transaction isolation level 0 SELECT * FROM suser holdlock</pre>	/* 0 уровень изоляции */
<pre>set transaction isolation level 0 SELECT * FROM suser holdlock at isolation read uncommitted</pre>	/*HOLDLOCK/NOHOLDLOCK/SHARED/READ PAST conflicts with AT ISOLATION LEVEL 0. */

Установка более строгих блокировок, чем текущий уровень

Если уровень изоляции 1 достаточен для большей части выполняемых запросов, но некоторые

запросы требуют более высоких уровней изоляции, можно задать для отдельных запросов более высокий уровень изоляции с помощью инструкций в операторе select:

- чтобы задать уровень 2, укажите инструкцию repeatable read;
- чтобы задать уровень 3, укажите инструкцию holdlock или at isolation serializable.

Ключевое слово holdlock делает разделяемую блокировку страницы или таблицы более строгой. Оно применяется:

- к разделяемым блокировкам;
- к таблице или представлению, для которых оно указано;
- во время выполнения оператора или транзакции, содержащей этот оператор.

Инструкция at isolation применяется ко всем таблицам, указанным в инструкции from, и только на время выполнения транзакции. Блокировки снимаются по завершении транзакции.

В транзакции ключевое слово **holdlock** указывает, что Adaptive Server должен удерживать разделяемые блокировки до завершения этой транзакции, а не снимать блокировку, как только таблица, представление или страница данных больше не будут нужны для команды, установившей блокировку. Монопольные блокировки всегда удерживаются Adaptive Server до конца транзакции.

Ключевое слово **holdlock** в следующем примере обеспечивает согласованность результатов двух запросов друг с другом:

```
begin transaction

select branch, sum(balance)
  from account holdlock
group by branch

select sum(balance) from account

commit transaction
```

Первый запрос устанавливает разделяемую блокировку таблицы account, чтобы никакая другая транзакция не могла изменять данные до выполнения второго запроса. Эта блокировка не снимается до завершения транзакции, содержащей команду с ключевым словом holdlock.

Установка менее строгих блокировок, чем текущий уровень

В противоположность ключевому слову holdlock, ключевое слово **noholdlock** не допускает, чтобы Adaptive Server удерживал разделяемые блокировки, установленные при выполнении запроса, независимо от текущего уровня изоляции транзакций. Ключевое слово noholdlock удобно

использовать в ситуациях, когда для транзакций нужно установить уровни изоляции по умолчанию, равные 2 или 3. Если ни для каких запросов в этих транзакциях не требуется удерживать разделяемые блокировки до конца транзакции, в этих запросах следует указать ключевое слово `noholdlock` для повышения производительности одновременного доступа.

Например, если для уровня изоляции транзакций установлено значение 3, что указывает запросу `select` удерживать блокировки до конца транзакции, эта команда снимает блокировку, после того как страница или строка будут пройдены сканированием:

```
select balance from account noholdlock
where acct_number < 100
```

Курсоры и блокирование

Методы блокировки курсоров похожи на другие методы блокировки в Adaptive Server. Для курсоров, объявленных как read only или без инструкции for update, Adaptive Server устанавливает разделяемую блокировку на странице данных, на которую указывает курсор.

При следующем извлечении строк из курсора (командой fetch) Adaptive Server устанавливает блокировку следующей страницы, позиция курсора перемещается на эту страницу, а блокировка предыдущей страницы снимается (если действующий уровень изоляции не равен 3).

Для курсоров, объявленных с инструкцией for update, при сканировании таблиц или представлений, указанных в этой инструкции, Adaptive Server устанавливает блокировки обновления для страниц.

Если в инструкции for update не указано ни одной таблицы и представления, для всех таблиц и представлений, указанных в инструкции from оператора select, устанавливаются **блокировки обновления**.

Блокировка обновления представляет собой специальный тип блокировки чтения, которая указывает, что команда, считывающая данные, может вскоре их модифицировать. Блокировка обновления позволяет устанавливать другие разделяемые блокировки на эту страницу, но не разрешает устанавливать другие блокировки обновления и монопольные блокировки. Когда через курсор происходит обновление или удаление строки, транзакция, содержащая команду, выполняющую это изменение, устанавливает монопольную блокировку. Любые монопольные блокировки, установленные транзакцией при обновлении через курсор, удерживаются до конца этой транзакции и не снимаются при закрытии курсора.

Это также относится к разделяемым блокировкам и блокировкам обновления для курсоров, которые определены с ключевым словом holdlock или уровнем изоляции 3.

Ниже описано поведение блокировок для курсоров при каждом уровне изоляции:

- На уровне 0 Adaptive Server не устанавливает никаких блокировок ни для каких страниц базовой таблицы, содержащих строку, на которую указывает курсор. Курсоры не устанавливают никаких блокировок чтения для сканирования, поэтому они не запрещают другим приложениям обращаться к тем же данным.
- На уровне 1 Adaptive Server устанавливает разделяемые блокировки или блокировки обновления на базовой таблице и листовых страницах индекса, содержащих строку, на которую указывает курсор. Страница остается заблокированной, пока текущая позиция курсора не изменится в результате выполнения операторов fetch.
- На уровне 2 или 3 Adaptive Server устанавливает разделяемые блокировки или блокировки

обновления на все базовые таблицы и листовые страницы индекса, которые читаются через курсор в данной транзакции. Adaptive Server удерживает блокировки до завершения транзакций; блокировки не снимаются, когда страница данных становится ненужной или курсор закрывается.

Использование ключевого слова **shared**

В операторе `declare cursor`, объявляющем обновляемый курсор (то есть содержащем инструкцию `for update`), можно указать, чтобы Adaptive Server устанавливал на страницах разделяемые блокировки (вместо блокировок обновления):

```
declare имя_курсора cursor
for select список_выборки
      from {имя_таблицы | имя_представления} shared
for update [of список_столбцов]
```

Это позволяет другим пользователям получить блокировку обновления для таблицы или базовой таблицы представления.

Ключевое слово `holdlock` можно указывать вместе с ключевым словом `shared` после имени любой таблицы или представления. Ключевое слово `holdlock` должно предшествовать ключевому слову `shared` в операторе `select`. Например:

```
declare authors_crsr cursor
for select au_id, au_lname, au_fname
      from authors holdlock shared
      where state != 'CA'
for update of au_lname, au_fname
```

Ниже описано, к чему приведут различные комбинации параметров `holdlock` и `shared` (оба указаны, один из них указан, ни один не указан) при объявлении обновляемого курсора:

- Если не указан ни один из этих параметров, курсор будет удерживать блокировку обновления для строки или страницы, содержащей текущую строку.
Другие пользователи не могут обновлять (ни через курсор, ни каким-либо другим образом) строку в позиции курсора (для таблиц с блокировкой строк данных) или любую строку на странице, содержащей текущую строку (для таблиц с блокировкой всех страниц или блокировкой страниц данных).
Другие пользователи могут объявлять курсор для тех же таблиц, что и курсор текущего пользователя, и могут считывать данные, но не могут устанавливать блокировку обновления или монопольную блокировку на текущую строку или страницу.
- Если указан параметр `shared`, курсор удерживает разделяемую блокировку на текущей строке или на странице, содержащей строку в текущей позиции курсора.

Другие пользователи не могут изменять (ни через курсор, ни каким-либо другим образом) текущую строку (или строки на текущей странице). Тем не менее они могут считывать строку или строки на странице.

- Если указан параметр `holdlock`, на всех извлеченных строках или страницах удерживаются блокировки обновления (если команда, извлекающая строки, не содержится в транзакции), или только для страниц, извлеченных после последней фиксации или последнего отката (если команда содержится в транзакции).

Другие пользователи не могут изменять (ни через курсор, ни каким-либо другим образом) строки в текущей позиции курсора (или страницы, содержащие эти строки).

Другие пользователи могут объявлять курсор для тех же таблиц, что и курсор текущего пользователя, но они не могут устанавливать блокировку обновления на строки в текущей позиции курсора или на страницы, содержащие эти строки.

- Если указаны оба параметра, курсор удерживает разделяемые блокировки для всех извлеченных строк или страниц (если команда, извлекающая строки, не содержится в транзакции), или для строк или страниц, извлеченных после последней фиксации или последнего отката транзакции. Другие пользователи не могут изменять (ни через курсор, ни каким-либо другим образом) строки в текущей позиции курсора (или страницы, содержащие эти строки).

DEADLOCKS

Deadlock – это ситуация, когда два или больше пользовательских процесса, уже установившие блокировку(и) на отдельной странице или таблице, одновременно пытаются получить доступ к заблокированным страницам или таблицам другого процесса. Транзакция с меньшим числом процессорных тиков откатывается и связанному с ней процессу посылается сообщение об ошибке.

Если процессы P1 и P2 стартуют одновременно, то они оба устанавливают исключительные блокировки в момент выполнения команды UPDATE. В дальнейшем, возникает deadlock, при котором оба процесса взаимно ожидают друг друга. ASE обнаруживает такие ситуации и выбирает в качестве жертвы тот процесс, который набрал меньшее число процессорных тиков.

После этого он откатывает транзакцию процесса-жертвы, посылает самому процессу сообщение об ошибке с номером 1205, второй процесс продолжает работу.

Ситуации с deadlock могут возникать при интерференции любых несовместимых между собой блокировок.

Пример :

P1	События	P2
begin transaction		begin transaction
update client set client_jf='F' where client_id='090004'	Процессы P1 и P2 запускаются одновременно. P1 получает исключительную блокировку на таблице client, P2 – на таблице account	update account set mfo_code='305017' where acc_num='2040...'
update account set mfo_code='305299' where acc_num='2040...'	P1 ждет пока P2 отпустит свои блокировки, в то же время P2 ждет P1 . DEADLOCK!	update client set client_jf='J' where client_id='090004'
commit transaction		commit transaction

Для получения информации о пользовательских процессах, заблокированных процессах и об активных транзакциях можно использовать ряд системных процедур:

sp_who	Возвращает информацию о процессах, запущенных на сервере. Если пользовательский процесс был заблокирован другим пользователем, то статус этого процесса будет показан как «lock sleep», для демонстрации факта того, что эта задача ожидает снятия блокировки какого-то объекта базы данных. В столбцах blk_spid и block_xloid находится ID процесса и транзакции, создавшего блокировки. Для получения информации о конкретном пользователе, эта процедура может быть запущена с его именем в качестве параметра. Иначе, sp_who выводит информацию о всех запущенных процессах на сервере.
sp_lock	Используется для получения информации о существующих блокировках. В качестве необязательного аргумента может выступать список идентификаторов процессов, запущенных на сервере.
sp_familylock	Результаты вызова sp_familylock без параметров идентичны результатам sp_lock; однако, sp_familylock позволяет получить отчет о блокировках семейства параллельных процессов, а не отдельного процесса
sp_transactions	Процедура sp_transactions без параметров выводит информацию о всех активных транзакциях, с ключевым словом «state» - об активных транзакциях в указанном состоянии, с «xid» и «хactname» выводит только имена активных транзакций в указанном состоянии, с «gtrid» - о транзакциях с указанным ID